Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Structures itératives, évaluation des conditions et prévention des boucles infinies

Synthèse exécutive : Ce cours aborde de manière approfondie l'automatisation de la répétition d'instructions en algorithmique et en langage C sans duplication de code, conformément au principe DRY (Don't Repeat Yourself). Il détaille le fonctionnement, la syntaxe exacte en C et les critères algorithmiques de choix entre les trois boucles fondamentales : 'while' (structure à pré-condition, exécutée de 0 à N fois), 'do-while' (structure à post-condition, exécutée au moins 1 fois, essentielle pour le contrôle de saisie) et 'for' (structure à itération déterminée regroupant initialisation, condition et mise à jour). Une attention particulière est portée sur la prévention des boucles infinies, la gestion rigoureuse de la variable de contrôle et le respect strict des normes de codage exigées à l'IUT d'Anglet (style Allman, tripartition du code et cartouche d'en-tête).

Utilité des structures itératives et principe DRY

En programmation algorithmique, la répétition d'instructions ne doit jamais résulter d'un copier-coller de lignes de code. Si l'on doit traiter 10 000 éléments d'un tableau ou réitérer une demande de saisie jusqu'à ce qu'une valeur soit valide, dupliquer le code détruit la maintenabilité du programme, alourdit inutilement le binaire exécutable et constitue une source majeure de bogues. Les structures itératives (ou boucles) permettent d'exécuter de façon automatisée et répétée un même bloc d'instructions tant qu'une condition logique (expression booléenne) reste vérifiée. Le point névralgique de la conception d'une boucle repose sur la gestion de sa fin. Une boucle robuste exige trois éléments indissociables : une initialisation correcte des variables de contrôle avant d'entrer dans la boucle, une condition de maintien/sortie claire, et une mise à jour explicite de la variable de contrôle dans le corps de la boucle à chaque itération. Si le corps de la boucle n'altère pas les variables testées dans la condition, celle-ci conserve sa valeur initiale Vraie : le programme entre alors dans une 'boucle infinie', gelant l'application.

- Principe DRY (Don't Repeat Yourself) : Interdiction fondamentale de dupliquer du code par copier-coller pour répéter une tâche similaire.
- Condition d'arrêt : Expression booléenne évaluée pour déterminer si la boucle se poursuit (VRAI) ou s'arrête (FAUX).
- Variable de contrôle / Compteur : Variable modifiée au sein du corps de la boucle pour faire évoluer la condition vers son état de sortie.
- Boucle infinie : État critique de blocage de l'exécutable provoqué par l'absence de mise à jour de la variable de contrôle, entraînant la saturation des ressources processeur.

Cas concret enseignant : *Métaphore du tapis roulant d'une usine d'emballage : tant qu'un colis est détecté par le capteur optique (condition vraie), le bras robotisé attrape le colis et l'emballe (instruction). Dès que le capteur ne détecte plus de colis ou que le quota fixé est atteint (condition fausse), le robot s'arrête. Sans capteur fonctionnel ou sans mise à jour du compteur de colis, le robot continuerait à tourner à vide indéfiniment jusqu'à provoquer une surchauffe et détruire la machine.*

La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est le choix algorithmique naturel lorsque le nombre exact d'itérations ne peut pas être déterminé à l'avance et dépend d'un événement dynamique externe (une saisie utilisateur, la lecture d'un fichier jusqu'à sa fin, ou la réception d'un signal réseau). En langage C, sa syntaxe stricte s'écrit : while (condition) { /* instructions */ } Puisque le contrôle s'effectue en amont du bloc, il est possible que les instructions internes ne s'exécutent jamais (0 itération) si la condition est fausse initialement. L'étudiant doit veiller à deux pièges fréquents : d'une part, l'omission de l'initialisation des variables de condition avant le while (conduisant à tester des valeurs indéterminées en mémoire) ; d'autre part, l'oubli de la modification de ces variables à l'intérieur de la boucle (provoquant une boucle infinie).

- Pré-conditionnement : L'expression booléenne est évaluée AVANT l'accès au bloc d'instructions.

• Plage d'itérations (0 à N) : Si la condition est fausse dès le premier contrôle, le corps de la boucle n'est jamais exécuté.

• Initialisation préalable obligatoire : Toute variable testée dans la condition du while doit posséder une valeur connue avant d'entrer dans la boucle.

Cas concret enseignant : *Exemple du contrôle de code PIN à un distributeur automatique de billets (DAB) : le système demande le code secret TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est strictement inférieur à 3. Si l'usager saisit le bon code du premier coup, la condition devient fausse immédiatement et le corps de la boucle n'est pas réexécuté.*

La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence de la boucle 'while', la structure 'do-while' garantit que le bloc d'instructions sera exécuté au moins une première fois avant que la condition logique ne soit testée. En langage C, sa syntaxe s'écrit : do { /* instructions */ } while (condition); Cette boucle est spécifiquement adaptée à la saisie sécurisée de données (validation de bornes numériques) et à la gestion de menus interactifs. En effet, il est impossible d'évaluer la validité d'une donnée saisie par un utilisateur avant de lui avoir au moins demandé de la saisir une première fois. Le piège de syntaxe majeur réside dans l'oubli du point-virgule final juste après `while (condition);`, erreur classique qui bloque immédiatement la compilation.

• Post-conditionnement : L'expression booléenne est évaluée APRÈS chaque exécution du bloc d'instructions.

• Garantie d'exécution minimale : La boucle s'exécute obligatoirement au moins 1 fois (1 à N itérations).

• Point-virgule final impératif : La syntaxe C exige un point-virgule ';' immédiatement après la parenthèse fermante du while : do { ... } while (condition);

Cas concret enseignant : *Contrôle de saisie d'une note d'examen (entre 0 et 20) : 'On ne peut pas vérifier si la note entrée par un utilisateur est valide avant qu'il n'ait tapé la valeur au clavier. Il faut exécuter la demande de saisie au moins une fois, ce qui fait du do-while la structure idéalement adaptée pour le contrôle de saisie.'*

La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en réunissant sur une ligne d'en-tête unique l'ensemble de la logique d'itération. En langage C, sa syntaxe s'écrit : for (initialisation ; condition ; incrémentation) { /* instructions */ } Le fonctionnement interne respecte un ordre strict : 1) Exécution unique de l'initialisation ; 2) Évaluation de la condition booléenne ; 3) Si la condition est Vraie, exécution du corps de la boucle ; 4) Exécution de l'incrémentation/décrémentation ; 5) Retour à l'étape 2. Bien que le C autorise l'omission de certaines parties dans l'en-tête (ex: `for(;;)` pour créer une boucle infinie), la norme d'écriture à l'IUT d'Anglet impose de renseigner explicitement les trois expressions pour garantir une lisibilité optimale.

• En-tête condensé : Centralise l'initialisation, la condition de maintien et la mise à jour sur une seule ligne.

• Nombre d'itérations déterminé : Structure recommandée lorsque le nombre exact de répétitions est connu avant d'entrer dans la boucle.

• Portée (scope) : Possibilité de limiter la durée de vie de la variable d'itération en la déclarant dans l'en-tête (standard C99).

Cas concret enseignant : *Métaphore des 5 tours de piste de l'athlète sur un stade : 'Avant de démarrer la course, vous savez que vous devez accomplir exactement 5 tours. Vous préparez votre compteur au tour 1 (initialisation), vous contrôlez que vous n'avez pas dépassé le 5e tour (condition), et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur (incrémentation).'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Sans l'incrémentation explicite de la variable de contrôle 'i' dans le corps de la boucle, l'expression logique (ex: i < N) conserve indéfiniment sa valeur initiale Vraie. Le programme entre alors dans une 'boucle infinie' : il réexécute indéfiniment le même bloc d'instructions sans jamais atteindre la condition de sortie, gelant l'exécutable et consommant 100% des ressources processeur.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, sur le plan algorithmique, toute boucle 'for(init; cond; inc)' est équivalente à une boucle 'while'. Il suffit de placer l'instruction d'initialisation avant l'entrée dans le while, la condition dans la parenthèse du while, et l'incrémentation à la toute fin du corps de la boucle. Cependant, on privilégie la boucle 'for' dès que le nombre d'itérations est connu à l'avance pour des raisons de lisibilité et de centralisation de la logique itérative.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers strictement positifs (1 + 2 + ... + N). Le programme doit impérativement sécuriser la saisie de N au moyen d'une boucle 'do-while' pour refuser toute valeur inférieure ou égale à 0, puis calculer la somme cumulative à l'aide d'une boucle 'for'. Le programme doit être rédigé selon la norme stricte de l'IUT d'Anglet (cartouche d'en-tête, tripartition CONSTANTES / VARIABLES / TRAITEMENTS, flèches de flot de données et style Allman).

Démarche de résolution :
Étape 1 : Déclaration des variables entières 'n', 'somme' et 'i' dans la section // VARIABLES. Étape 2 : Implémentation du contrôle de saisie avec la structure do-while : la condition (n <= 0) maintient l'utilisateur dans la boucle tant qu me la valeur saisie est négative ou nulle. Étape 3 : Initialisation explicite de l'accumulateur 'somme' à 0 avant d'entamer le calcul. Étape 4 : Exécution de la boucle 'for' allant de i = 1 jusqu'à i = n avec un pas de +1, en ajoutant 'i' à 'somme' à chaque tour. Étape 5 : Affichage de la valeur finale via printf et retour de la fonction main.

```c
/*
 * En-tête de fichier
 * Titre : Calcul de la somme des N premiers entiers positifs
 * Auteur : Matéi de Saint Leger
 * Date : 14/11/2026
 * Objectif : Saisie sécurisée de N (>0) avec do-while et calcul de la somme avec for
 */

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    // CONSTANTES

    // VARIABLES
    int n;     // Nombre d'entiers à cumuler (saisi par l'utilisateur)
    int somme; // Accumulateur de la somme cumulative
    int i;     // Compteur de boucle pour l'itération

    // TRAITEMENTS
    // Saisie sécurisée du nombre N (do-while)
    do
    {
        // (écran) << afficher message de demande
        printf("Entrez un nombre entier strictement positif N (>0) : ");
        // (clavier) >> saisir >> n
        scanf("%d", &n);

        if (n <= 0)
        {
            // (écran) << afficher message d'erreur
            printf("Erreur : la valeur doit etre strictement superieure a 0.\n");
        }
    } while (n <= 0);

    // Initialisation du cumul de la somme
    somme = 0;

    // Calcul de la somme des N premiers entiers (for)
    for (i = 1; i <= n; i++)
    {
        somme += i;
    }

    // Affichage du résultat final
    // (écran) << afficher résultat
    printf("La somme des %d premiers entiers est : %d\n", n, somme);

    return 0;
}
```

Résultat : Pour une saisie utilisateur N = 5 : - La boucle do-while rejette toute saisie <= 0 (ex: -3) et redemande la valeur. - Une fois N = 5 validé (5 <= 0 est FAUX), la boucle for effectue 5 itérations : * i = 1 : somme = 0 + 1 = 1 * i = 2 : somme = 1 + 2 = 3 * i = 3 : somme = 3 + 3 = 6 * i = 4 : somme = 6 + 4 = 10 * i = 5 : somme = 10 + 5 = 15 Le programme affiche correctement : 'La somme des 5 premiers entiers est : 15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre une boucle 'while' et une boucle 'do-while' concernant le nombre minimum d'itérations ?

R : La boucle 'while' effectue son test à pré-condition (avant l'entrée), ce qui signifie que son corps peut s'exécuter de 0 à N fois (0 itération si la condition est fausse dès le départ). La boucle 'do-while' effectue son test à post-condition (après le corps), ce qui garantit au minimum 1 exécution (de 1 à N itérations).

Q : Que se passe-t-il si la condition d'une boucle 'for' est fausse dès la toute première évaluation ?

R : L'instruction d'initialisation est exécutée une fois, puis la condition est évaluée à FAUX. Le corps de la boucle 'for' ainsi que l'instruction d'incrémentation sont complètement ignorés, et le programme poursuit directement son exécution après la boucle (0 itération du corps).

Q : Quel est l'oubli de syntaxe classique qui provoque une erreur de compilation immédiate sur la boucle 'do-while' en C ?

R : L'oubli du point-virgule final ';' immédiatement après la parenthèse fermante de l'expression while : `do { ... } while (condition);`. Sans ce point-virgule, le compilateur C génère une erreur de syntaxe.